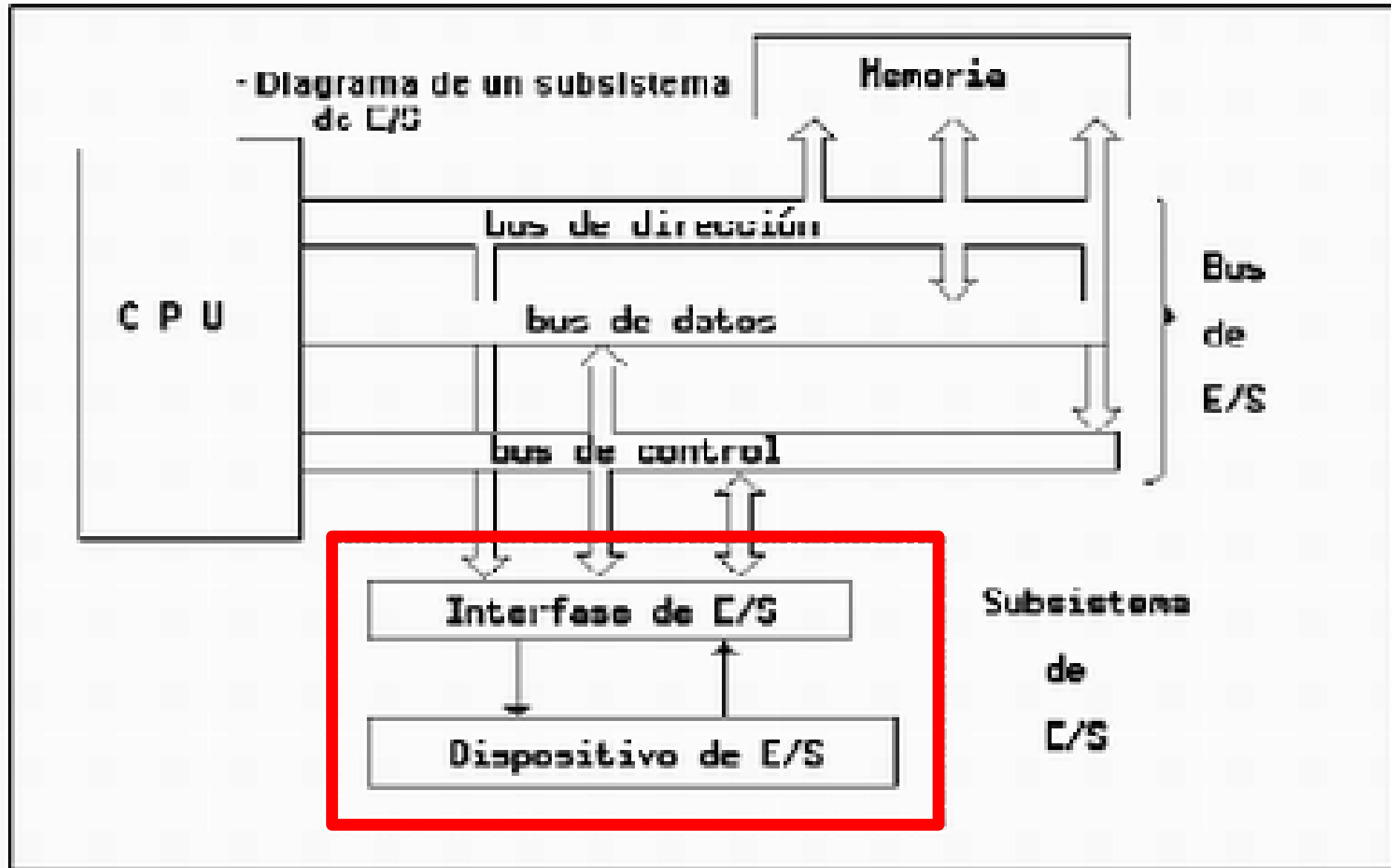


# **CATEDRA DE ARQUITECTURA DE COMPUTADORAS**

**INTERFAZ DE ENTRADA / SALIDA**

# INTERFAZ DE E/S



# INTRODUCCIÓN

- Un subsistema de E/S consiste en interfases de E/S y **dispositivos periféricos**.
- **La interfaz de E/S** controla la operatoria de los dispositivos conectados a ella. Las operaciones de control (por ejemplo rebobinado, posicionamiento, etc.) se arrancan mediante comandos emitidos por la CPU.
- **Periférico:** Es cualquier elemento capaz de realizar un intercambio de datos con la CPU o la Memoria.

# **FUNCIONES DE LA INTERFAZ DE E/S**

- ✓ **Almacenar los datos y realizar las conversiones que se le requieran.**
- ✓ **Detectar errores en la transmisión y ser capaz de reiniciar la transacción en estos casos.**
- ✓ **Testear, arrancar y detener el dispositivo según las directivas impartidas por la CPU.**
- ✓ **Realizar el Control y Temporización del Flujo de Datos.**
- ✓ **Establecer la Comunicación con la CPU**
  - **Decodificación de comandos**
  - **Intercambio de los datos de E/S con la CPU**
  - **Informar el estado del dispositivo**
  - **Reconocimiento de la dirección del dispositivo**

# PROBLEMÁTICA

La cantidad y variedad de dispositivos es muy grande, y el **Sistema de E/S** debe proveer soluciones para los siguientes problemas:

- ✓ **Direccionamiento:**

Coexisten varios periféricos, y las operaciones de lectura, escritura y control se deben realizar en forma independiente.

- ✓ **Velocidades diferentes:**

Los periféricos pueden tener un rango de velocidades muy amplio.

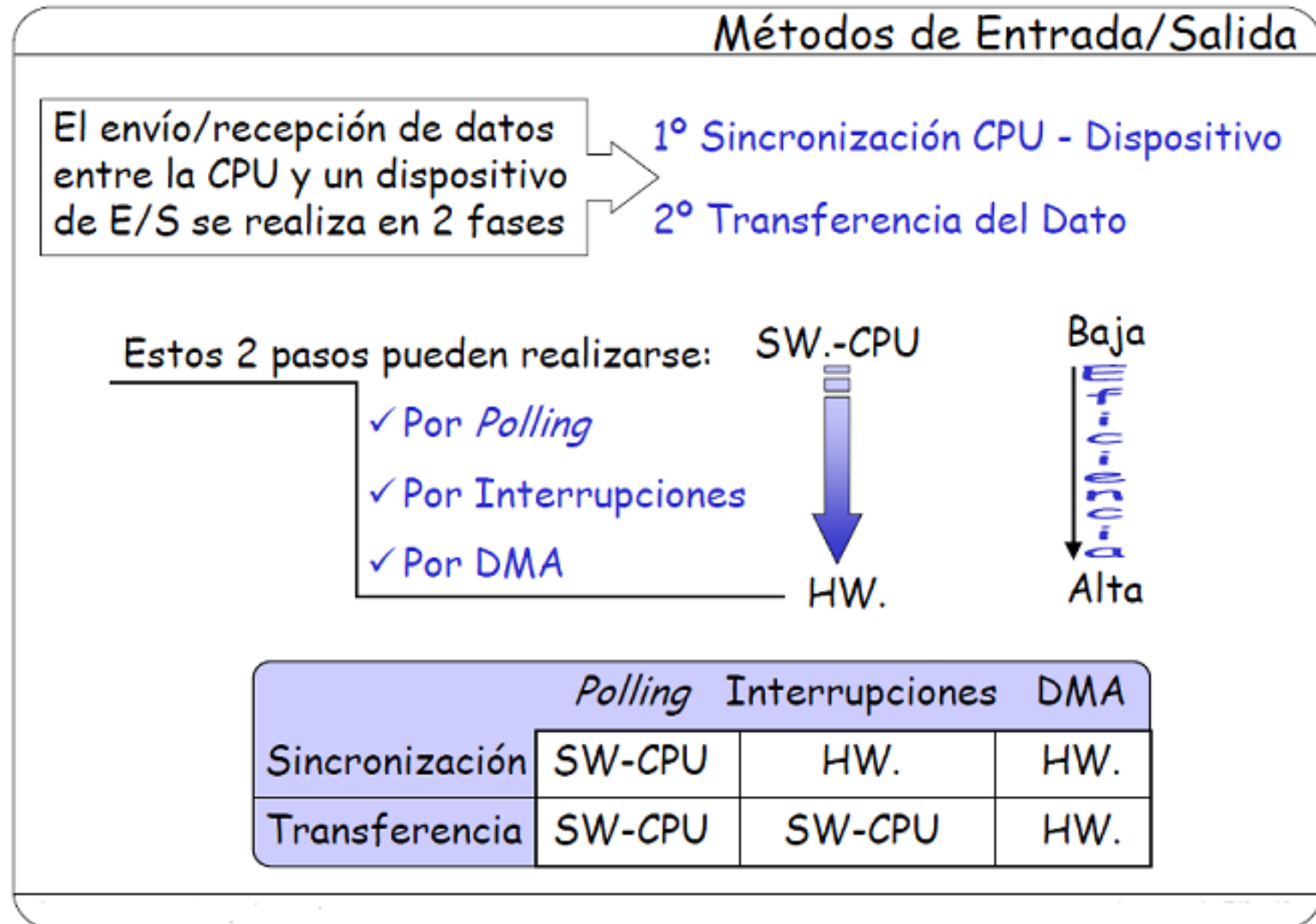
- ✓ **Acceso lectura/escritura:**

Existen periféricos de sólo lectura, de escritura y de lectura/escritura.

# TRANSFERENCIA ELEMENTAL Y OPERACIONES DE E/S

- ✓ Los modos de operación con la E/S son diferentes en función de que se trate de:
- ✓ **Transferencia Elemental:** Intercambio de una única unidad de información con el periférico.
  - ✓ Es necesario establecer una comunicación física entre el periférico y el resto del sistema y a continuación llevar a cabo el control del periférico mediante la consulta y modificación de su estado.
- ✓ **Operación de E/S:** Transferencia de un conjunto de datos.
  - ✓ Aparte de contar el número de datos hay que sincronizar las velocidades de los periféricos con el resto del sistema, donde difiere del caso anterior y detectar posibles errores mediante códigos de paridad o polinomiales, y si es preciso repetir la transmisión.

# FORMAS DE REALIZAR LA TRANSFERENCIA DE E/S



# Formas de Realizar la Transferencia de E/S

El envío o recepción de un dato hacia / desde un dispositivo de E/S se debe realizar en dos pasos:

## 1 - SINCRONIZACION CPU-DISPOSITIVO

Ambos realizan tareas distintas y a velocidades distintas

## 2 - TRANSFERENCIA DEL DATO

Existe tres métodos principales de E/S:

➤ **Pooling (Encuesta):** (Es la forma mas sencilla)

El sincronismo entre CPU y dispositivos se consigue sondeando a cada uno de ellos.

## CPU

1. Leer Registro de Estado del controlador
3. Comprobar estado.  
Si no preparado, ir a Paso 1
4. Escribir dato en el reg. de datos del controlador
8. Si hay más datos para enviar, ir a Paso 1

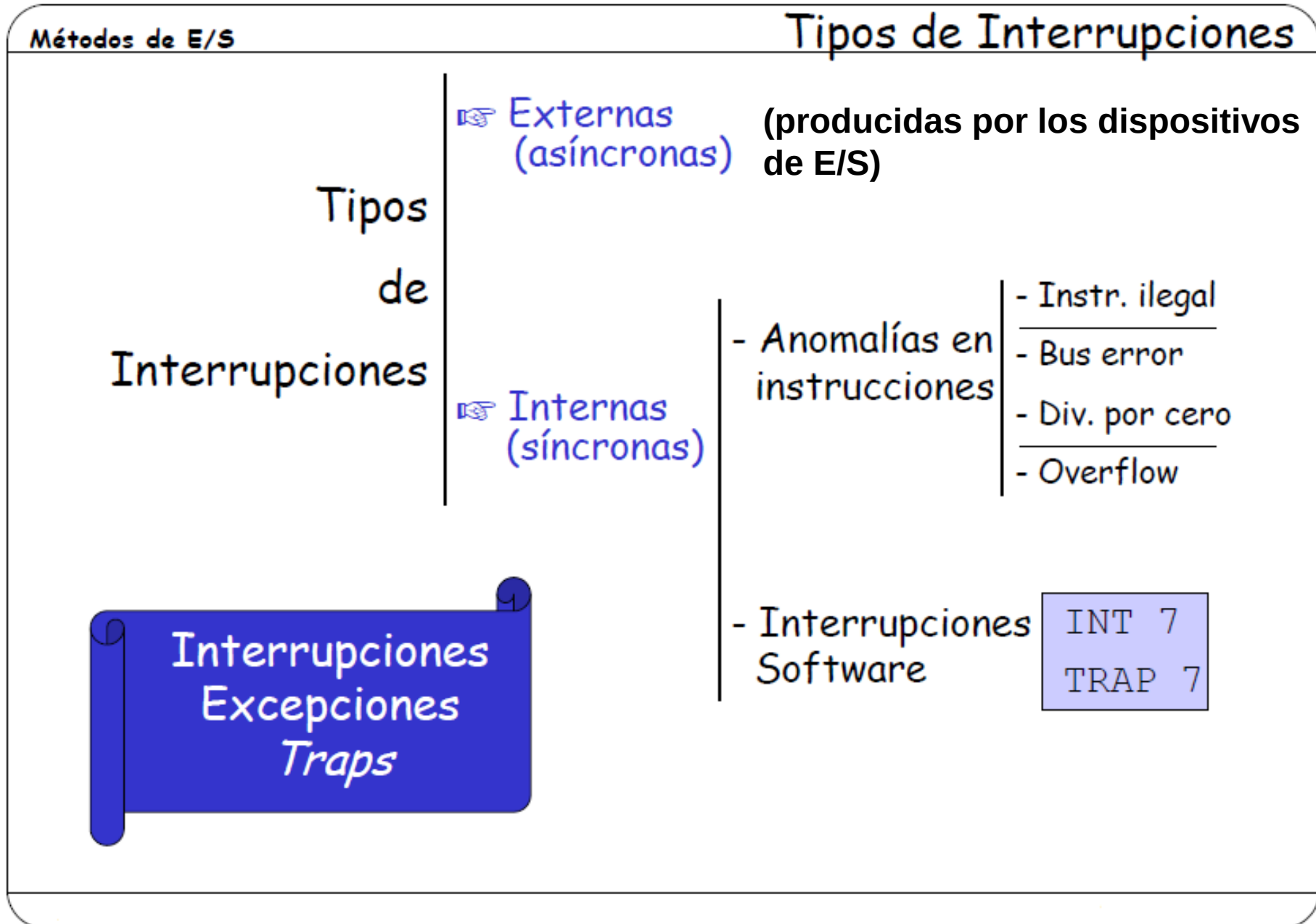
## Controlador

2. Enviar estado
5. Aceptar dato y poner estado a "No preparado"
6. Esperar a que el dato se escriba en el dispositivo
7. Poner estado a "Preparado"

## PROBLEMAS

- ✓ Pérdida de Tiempo (Espera activa)
- ✓ Lentitud (si no se atiende pronto)

## ➤ Por Interrupciones:



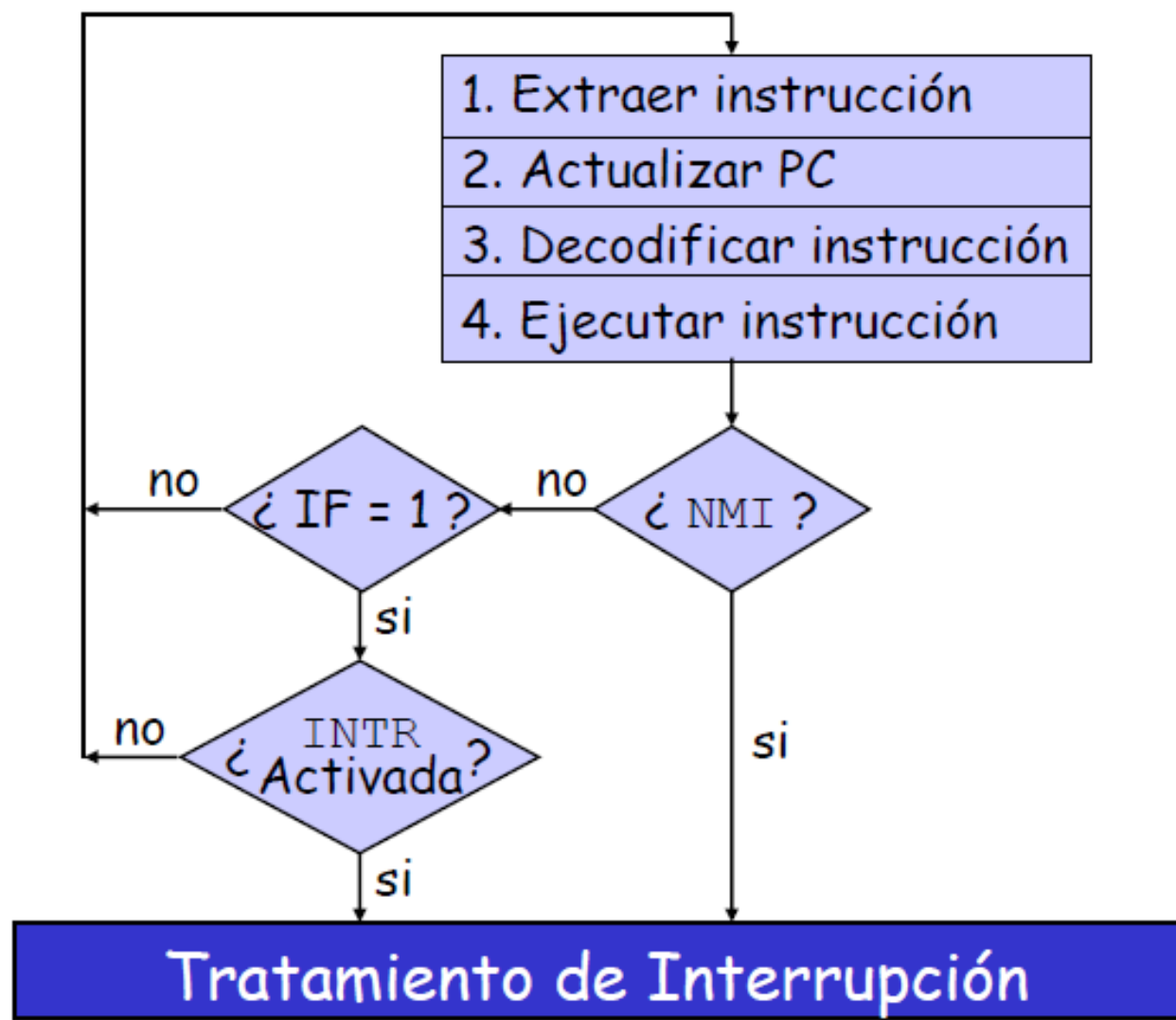
¿Siempre hay que atender a las interrupciones?

Interrupciones

No  
Enmascarables

- *Reset* (RESET)
- Error de Bus (BERR)
- NMI (NMI)

Enmascarables*ENABLE**DISABLE**Flag en Reg. Estado*

Aceptación

## ➤ **Por DMA (Direct Memory Access):**

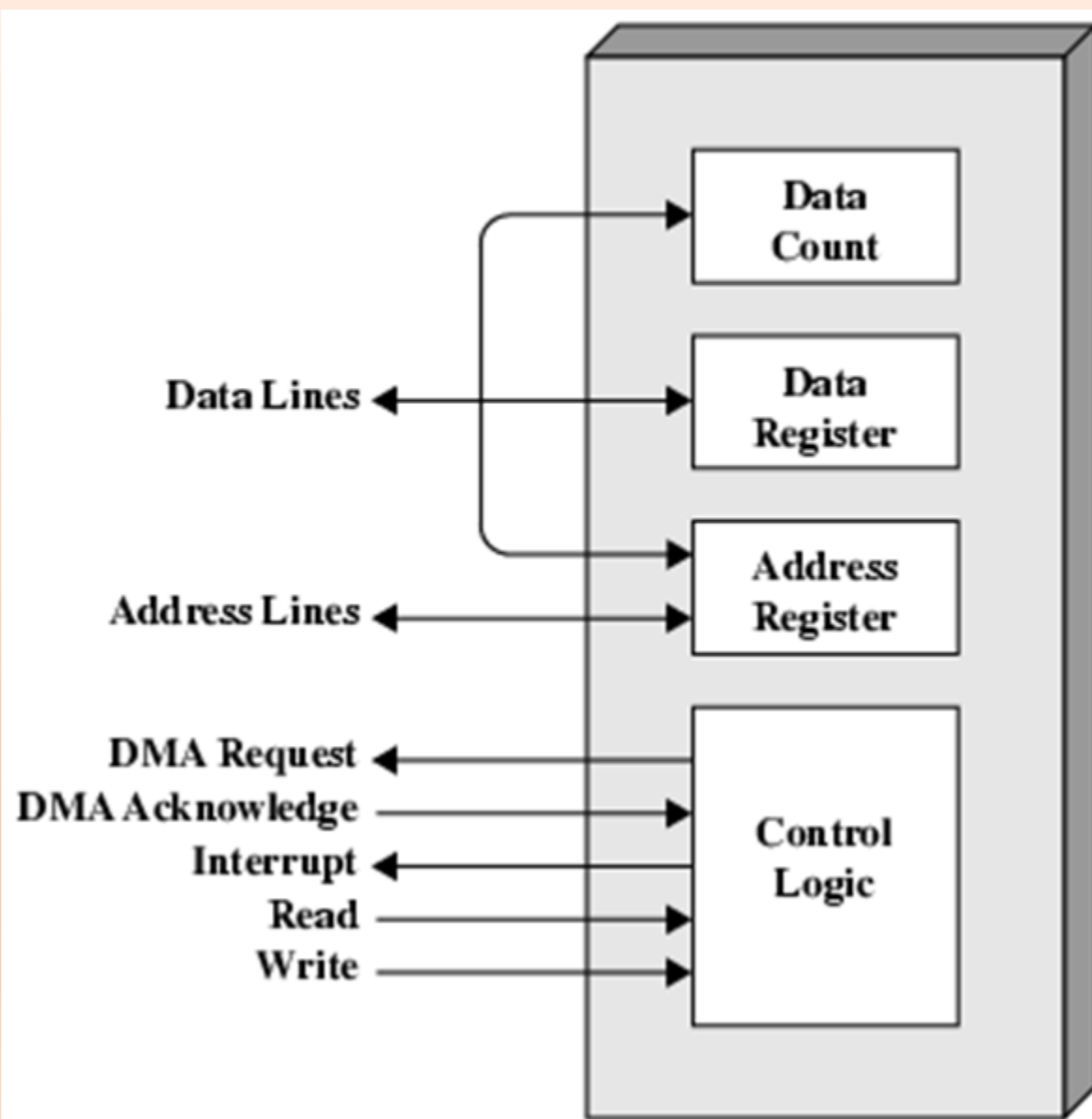
**El periférico se comunica directamente con la MP sin intervención de la CPU, esto acelera el proceso.**

**Requiere de un CONTROLADOR DMA (especialmente adecuado para transferencias de Bloques de Datos)**

**Para poder realizar operaciones de E/S por DMA el controlador debe conocer:**

- ✓ **Dirección del periférico**
- ✓ **Rango de memoria donde se realiza la operación**
- ✓ **Tipo de operación a Realizar (Lectura o Escritura)**
- ✓ **Número de operaciones elementales que van a integrar esa operación de E/S**

## Diagrama de controlador DMA



## MODO BLOQUE (BURST)

Cuando el **DMA** toma posesión del bus, se realiza la transferencia de un bloque de un gran número de palabras de datos, y la **CPU** queda en estado de **HALT** o **STOP** al no poder utilizar los buses.

Al finalizar la transferencia, el controlador de **DMA** envía una interrupción a la **CPU**, con lo que esta sale del estado **STOP**, atiende la interrupción y continúa la ejecución del programa que se interrumpió al ceder el bus al **DMA**.

**Se utiliza con dispositivos de memoria secundaria**, como los discos, donde las transferencias de datos no se pueden detener o ralentizar (pérdida de información).

**Transferencias de datos más rápidas** a costa de tener la **CPU detenida** (sin ejecutar instrucciones) mientras se realiza la transferencia.

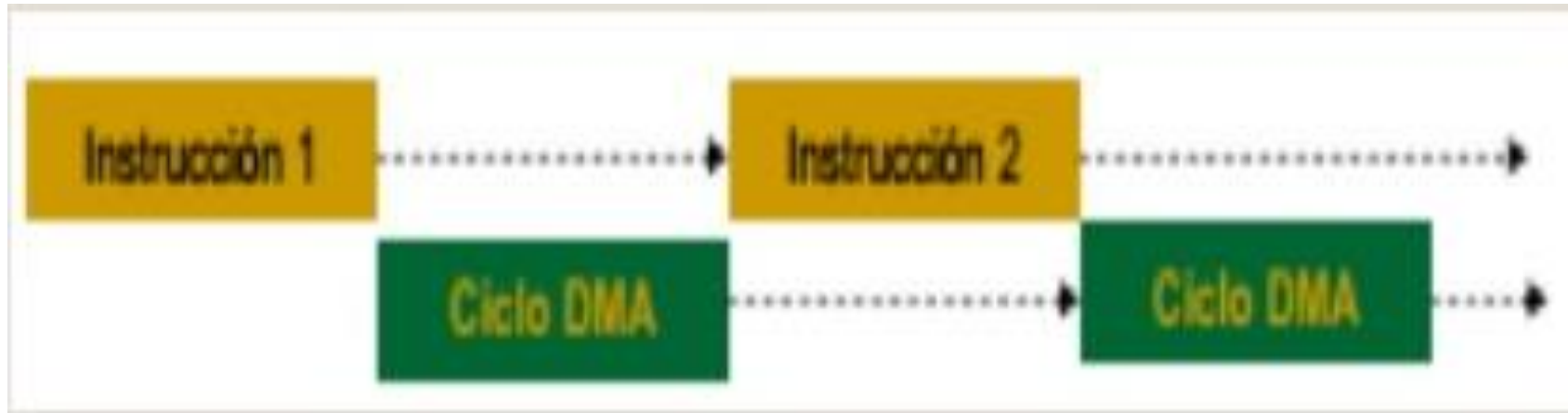
## ROBO DE CICLO

Alternativa al modo Bloque, Cuando se le **cede el control del bus al DMA**, se le permite que lo utilice para **transferir una única palabra de datos**, una vez finalizada la transferencia debe devolver el control del bus al **CPU**.

Con este sistema, la transferencia de bloques de datos tiene que realizarse mediante **una serie de ciclos de bus de DMA** intercalados con **ciclos de bus de la CPU**.

Este modo reduce la velocidad de transferencia respecto al **Modo Bloque**, pero también reduce la interferencia que se le causa a la **CPU**.

## ROBO DE CICLO

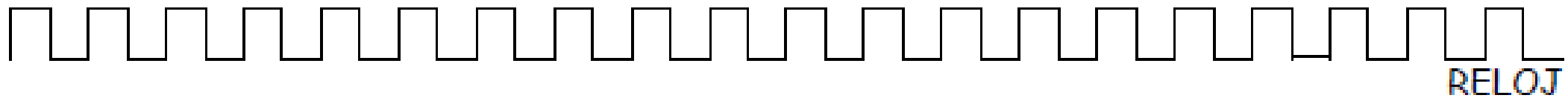


**Ambas actividades**, la del procesador y la del controlador, **se realizan en paralelo**, aunque es evidente que ambas resultan ralentizadas al tener que **repartirse** el tiempo de posesión del bus.

## BUS TRANSPARENTE

La interferencia ocasionada por los métodos anteriores puede eliminarse completamente si se diseña una interfaz de **DMA** que **solamente utilice ciclos de bus cuando la CPU no lo necesita**, ya que esta no requiere la posesión del bus el 100% del tiempo de ejecución de una instrucción.

El bus puede quedar **libre** cuando la CPU está **decodificando una instrucción**, cuando está **calculando la dirección de los operandos o del resultado**, o cuando está **utilizando la ALU**.



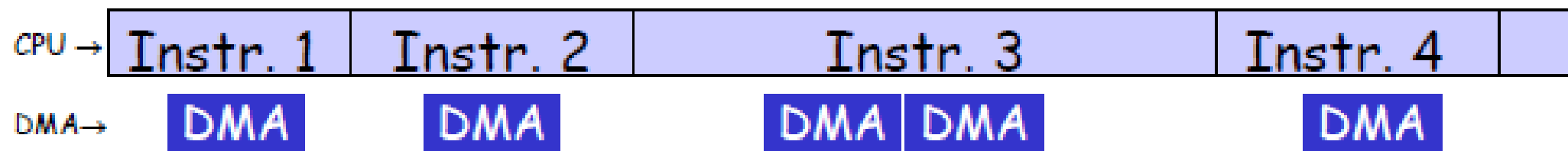
## Transferencia de Bloque



## Robo de Ciclo



## Bus Transparente



# La Evolución del Funcionamiento de la E/S

1. **La CPU controlaba directamente al periférico.** Esta situación se observa en los dispositivos controlados por microprocesadores.
2. **Se añade un controlador o módulo de E/S.** La CPU utiliza E/S programada sin interrupciones (**Polling**). De esta forma, la CPU se independiza de los detalles específicos de las interfaces de los dispositivos externos.
3. Se utiliza la misma configuración de la etapa 2, pero **ahora se emplean interrupciones**. La CPU no necesita esperar a que se realice la operación de E/S, incrementándose la eficiencia.
4. El módulo de E/S tiene acceso directo a memoria a través del **controlador DMA**. Permite transferir un bloque de datos hacia o desde memoria sin utilizar CPU.
5. El módulo de E/S se mejora haciendo que **se comporte como un procesador en si**, con un repertorio especializado de instrucciones orientados a las operaciones de E/S.
6. **El módulo de E/S tiene una memoria local propia y es, de hecho, un computador en si mismo.** Con esta arquitectura, permite controlar un conjunto grande de dispositivos con la mínima intervención de la CPU.

# E/S - Conclusiones

En toda transferencia se utilizan señales de control, datos y dirección, esta vez sobre los buses de E/S.

Las señales de control y tiempo se utilizan para regular la transferencia elemental, indicando como y cuando debe ocurrir.

O sea que permiten contestar las siguientes preguntas:

- ✓ Puede producirse una transferencia ahora?
- ✓ Que tipo de transferencia se va a realizar?
- ✓ Cuando comienza?
- ✓ Cuando termina?
- ✓ Se realizó con éxito?

**Estas preguntas nos permiten deducir que las funciones de la "lógica o inteligencia" de un bus de E/S son:**

- ✓ Comunicarse con el periférico y el sistema CPU-memoria.**
- ✓ Controlar la temporización durante la transferencia.**
- ✓ Almacenar temporalmente bits para "paliar" la diferencia de velocidad entre emisor y receptor.**
- ✓ Detectar si se produjeron errores durante la transferencia.**

Cuando la actividad del bus de E/S es “síncronica” se utilizan señales de **clock** que regulan la transferencia, por lo tanto, la **frecuencia de clock** es un parámetro a tener en cuenta para la velocidad de transferencia.

Un bus síncronico requiere que los dispositivos conectados a él, que se encuentren “**sintonizados**” a esa frecuencia y que todas las actividades se produzcan en intervalos de tiempo fijos.

Cuando la actividad del bus es “asíncronica”, las señales de **clock** no regulan su operación.

En este caso, su velocidad depende de los tiempos de los dispositivos conectados a ellos.

Tipos de señales diferentes comunes a la mayoría de los buses de E/S para **"controlar"** la transferencia:

- ✓ **Señal de clock**, que permite sincronizar el ciclo del bus con la operación de la CPU (CLK).
- ✓ **Señal que puede habilitar un tiempo de espera.**
- ✓ **Señal de lectura o escritura** a un dispositivo de E/S o a la memoria (IOR o IOW).
- ✓ **Señales de interrupción a la CPU (IRQn).**
- ✓ **Señales de reconocimiento (aceptación o requerimiento)**, que permiten el dialogo de los dispositivos durante la transferencia (p. ej., la técnica de handshaking o "apretón de manos").
- ✓ **Señal de bus cedido u ocupado.**

Las Señales de Dirección permiten representar la dirección del emisor y el receptor ( $A_n$  = address “sub” n).

Cada dispositivo de E/S tiene **asociada una combinación binaria**.

Por ejemplo, cuando se envía una transferencia al receptor, este "**reconoce**" su dirección tomando los bits  $A_n$  del bus y decodificándolos.

O sea que las **señales de dirección** permiten contestar las preguntas:

**¿De quien? y ¿A quien?**

Las Señales de Datos representan los bits del mensaje que se debe transferir

$$(D_n = \text{data "sub" } n)$$

El “**ancho de bus**” o bien la cantidad de líneas afectadas a la transferencia elemental dan una medida de potencial de trabajo:

A mayor cantidad de bits transferidos en paralelo => Mayor capacidad de transferencia.

Las unidades de comunicación con el bus son unidades hardware, que actúan de intermediarias en la comunicación:

**“CPU - memoria interna” => “controlador – periférico”**

Resuelven, fundamentalmente, el problema de disparidad en los tiempos de operación entre las unidades que conectan.

Se pueden agrupar en:

- ✓ **Interfaces paralelo.**
- ✓ **Interfaces serie.**
- ✓ **Controlador DMA (o sólo DMA's).**
- ✓ **Canal o procesador de entrada-salida (Input Output Processor o IOP).**

- ✓ La elección del tipo de intermediario tiene que ver con las características de cada dispositivo que se ha de conectar.
- ✓ En un mismo sistema pueden existir todos o algunos de ellos.
- ✓ A su vez, cada intermediario puede estar asociado con más de un dispositivo.

## ❖ Controladores

El término controlador se utiliza en gran medida para definir cualquier unidad hardware que "**gobierne**" a otra.

Un controlador de periférico es un dispositivo asociado en forma directa al periférico, que puede estar físicamente integrado a él, o bien separado de éste, y está constituido por:

- ✓ Un **buffer interno (memoria RAM)**, que permite el almacenamiento de la información que "viaja" desde o hacia el soporte.
- ✓ Una **lógica de control**, que interpreta comandos del periférico, genera señales para su ejecución y gobierna así la unidad.

Ej: Controlador de Disco, Controlador de Impresora, etc.

## ❖ Controladores

Su función se puede resumir en, al menos, dos operaciones fundamentales:

- ✓ Aislar el software de servicio de entrada/salida, que se "ocupa" de la transferencia de los detalles específicos del hardware del periférico y los convierte en invisibles.
- ✓ Compatibilizar la velocidad del periférico respecto de la del resto del sistema.
- ❑ Otros controladores, cuya función permite gobernar actividades del sistema, liberan a la CPU de ciertas tareas que pueden llevarse a cabo de manera independiente.
- ❑ Cuando un controlador se involucra con una transferencia, la CPU recibe parámetros de un programa que ejecuta a través de una interfaz, este programa forma parte de la administración de entrada/salida y se denomina **manejador de dispositivo (device driver)**.

## ❖ Interfaces

Una interfaz es un hardware que actúa de nexo entre un periférico o un adaptador y el bus. Sirve principalmente para adecuar las señales y preparar la transferencia elemental basada en un protocolo (p.ej., un byte).

No tiene capacidad suficiente para tomar la responsabilidad de la transferencia completa (**bloque**), la CPU o el canal son los que asumen el control de una transferencia completa.

### Tipos:

✓ *Interfaz Paralela*

✓ *Interfaz Serie*

# Interfaz Paralela



**Puerto paralelo de impresora**

- **Interfaz Paralela**

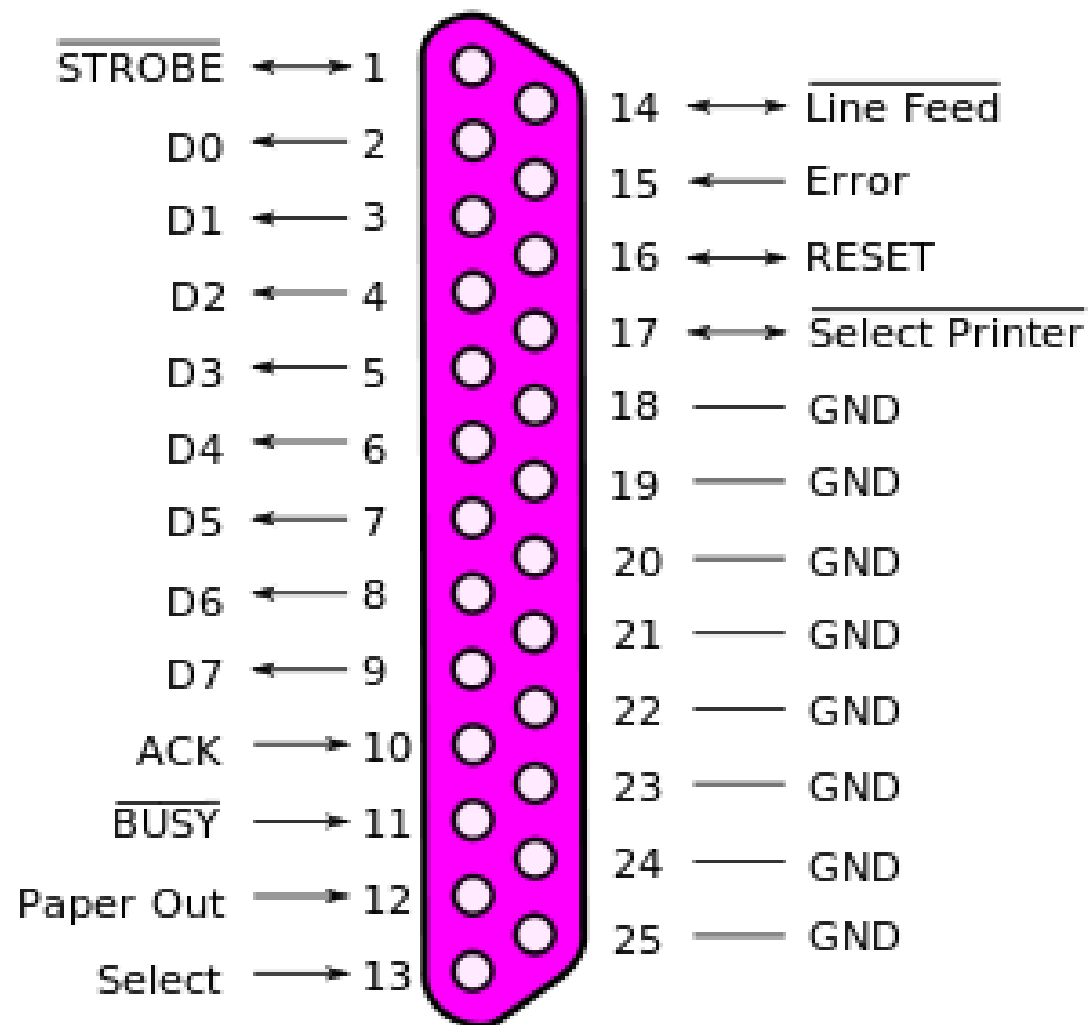
Es un dispositivo hardware que permite el control de la transferencia “en paralelo” entre el bus de sistema y un periférico.

Está asociada a una lógica de direccionamiento que permite establecer que esa interfaz ha sido seleccionada.

La interfaz cuenta con registros denominados **ports**. Un **port** está dividido en partes, por ejemplo, registro de dato (o port de dato) y registro de control (o port de control).

El registro de control tiene una función doble:

- ✓ En primer lugar, recibe un comando que puede enviar al periférico.
- ✓ En segundo lugar, recibe señales de control de estado; o sea que cada uno de sus bits se puede interpretar como indicadores de estado o flags que informen acerca de la transferencia.



Distribución de entradas y salidas en el conector DB25 para el puerto paralelo.

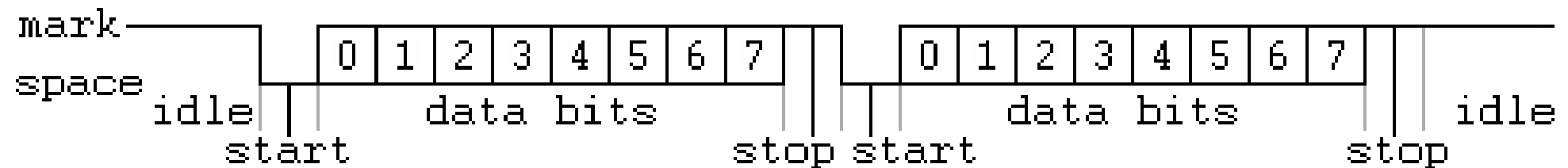
Los pines del puerto paralelo con conector DB25 son:

| Pin No (DB25) | Pin No (36 pin) | Nombre de la señal | Dirección | Registro - bit | Invertidas |
|---------------|-----------------|--------------------|-----------|----------------|------------|
| 1             | 1               | Strobe             | E/S       | Control-0      | Si         |
| 2             | 2               | Data0              | Salida    | Data-0         | No         |
| 3             | 3               | Data1              | Salida    | Data-1         | No         |
| 4             | 4               | Data2              | Salida    | Data-2         | No         |
| 5             | 5               | Data3              | Salida    | Data-3         | No         |
| 6             | 6               | Data4              | Salida    | Data-4         | No         |
| 7             | 7               | Data5              | Salida    | Data-5         | No         |
| 8             | 8               | Data6              | Salida    | Data-6         | No         |
| 9             | 9               | Data7              | Salida    | Data-7         | No         |
| 10            | 10              | Ack                | Entrada   | Status-6       | No         |
| 11            | 11              | Busy               | Entrada   | Status-7       | Si         |
| 12            | 12              | Paper-Out          | Entrada   | Status-5       | No         |
| 13            | 13              | Select             | Entrada   | Status-4       | No         |
| 14            | 14              | Linefeed           | E/S       | Control-1      | Si         |
| 15            | 15              | Error              | Entrada   | Status-3       | No         |
| 16            | 16              | Reset              | E/S       | Control-2      | No         |
| 17            | 17              | Select-Printer     | E/S       | Control-3      | Si         |
| 18-25         | 18-25           | Tierra             | -         | -              | -          |

# Interfaz Serie



**Puerto Serie**



- **Interfaz Serie**

Es un dispositivo hardware que permite el control de la transferencia de bits “en serie” entre el bus y un dispositivo de E/S.

Está asociado con una lógica de direccionamiento que permite establecer que esa interfaz fue seleccionada por el procesador para la transferencia.

Los registros de la interfaz constituyen el denominado puerto serie.

La interfaz se inicializa colocando un byte en su registro de control, que establece el tipo de transferencia.

En el **modo salida** la interfaz recibe un dato enviado por la CPU a través del bus de datos; este dato se almacena en un registro de recepción relacionado con un registro de desplazamiento, que provoca la serialización de la unidad de información para transferir hacia el dispositivo.

- **Interfaz Serie**

En el **modo entrada** otro registro de desplazamiento recibe la información bit tras bit desde el dispositivo y la coloca a disposición de la CPU cuando la unidad de información está completa en el registro.

La interfaz serie acepta las modalidades sincrónica y asincrónica; sin embargo, cuando los datos se transmiten en una línea desde un punto al otro, se produce un retardo de la señal en el medio de transmisión, que provoca incertidumbre acerca de dónde termina un bit y dónde empieza el otro.

Además, otra desventaja con respecto a la transmisión paralela es que la transmisión serie crea la dificultad de la delimitación de los caracteres; esto es, al enviarse los bits uno tras otro, no se sabe dónde empieza y dónde termina un carácter.

La técnica utilizada para resolver el problema depende de que la transmisión sea sincrónica o asincrónica.